

Лабораторная работа № 3

Задача: Минимизировать конечный автомат Мили алгоритмом Пола-Ангера. Вывести минимизированный автомат на экран.

Важно:

1. Лабораторная работа выполняется на языке C++ в среде Microsoft Visual Studio с использованием объектно ориентированного подхода. Результатом выполненной работы является программа, выполняющая требуемую задачу.

2. При выполнении лабораторных работ оцениваются следующие критерии:

- написанная студентом программа и демонстрация её работы; оценивается корректность работы программы, соблюдение требований к программе;

- точность ответов на контрольные вопросы по теме лабораторной работы;

- самостоятельная работа при написании программ.

Задание:

f

2	3	3	-	-
-	5	4	1	-
3	2	-	2	1
2	-	5	-	-

g

1	1	1	-	-
-	2	2	2	-
-	1	-	-	2
1	-	1	-	-

Важно: Лабораторная работа выполняется в четыре фазы.

Фаза 1 – формирование таблицы предварительной совместимости.

Фаза 2 – формирование таблицы полной совместимости.

Фаза 3 – вычисление классов максимальной совместимости.

Фаза 4 – формирование минимизированного автомата.

Фаза 1. Сформировать таблицу предварительной совместимости.

3. Описать абстрактный класс таблицы совместимости `CompTable`.

4. Описать класс пары состояний `SCouple`. Полями класса должны являться номера двух состояний.

5. Описать производный класс от класса `CompTable` таблицы предварительной совместимости `PreCompTable`.

Конструктор класса должен принимать в качестве входных параметров таблицы переходов и выходов автомата Мили в *определённом* виде и формировать таблицу предварительной совместимости с объектами типа `std::vector<SCouple>`.

Особенности реализации классов `CompTable`, `PreCompTable`, `SCouple` продумываются самостоятельно.

Фаза 2. Сформировать таблицу полной совместимости.

Описать производный класс от класса `CompTable` таблицы полной совместимости `FCompTable`.

Конструктор класса должен принимать в качестве входного параметра таблицу предварительной совместимости типа `PreCompTable` и формировать таблицу полной совместимости с объектами типа `std::vector<SCouple>`.

Особенности реализации класса `FCompTable` продумываются самостоятельно.

Фаза 3. Вычислить классы максимальной совместимости.

Описать класс множества классов максимальной совместимости `SMaxComp`.

Объект класса может хранить множество классов максимальной совместимости в виде поля типа `std::vector<std::vector<int>>`.

Конструктор класса должен принимать в качестве входного параметра таблицу полной совместимости FCompTable и вычислять множество классов максимальной совместимости.

Класс SMaxComp должен содержать метод void Sort(), выполняющий сортировку классов максимальной совместимости по двум условиям:

- по уникальности вхождения номера состояния в класс максимальной совместимости;
- по убыванию количеству состояний в классе максимальной совместимости.

Пример сортировки по двум условиям:

Дано множество классов максимальной совместимости:

$S_0S_1S_4$, $S_0S_1S_2S_3$, $S_1S_2S_3$, $S_0S_1S_2$, $S_0S_2S_3$, S_1S_2 , S_2S_3 .

После сортировки получим следующий порядок классов максимальной совместимости:

$S_0S_1S_4$, $S_0S_1S_2S_3$, $S_0S_1S_2$, $S_0S_2S_3$, $S_1S_2S_3$, S_1S_2 , S_2S_3

Фаза 4. Формирование минимизированного автомата.

1. Произвести сортировку в объекте класса SMaxComp с помощью метода Sort();
2. Последовательно выбирая состояния из объекта класса SMaxComp сформировать таблицу переходов минимизированного автомата. При доопределении состояний внутри таблицы приоритет отдаётся текущему классу максимальной совместимости, далее следующим классам максимальной совместимости из отсортированного множества классов максимальной совместимости в порядке сортировки.
3. Сформировать таблицу выходов минимизированного автомата.
4. Вывести таблицы переходов и выходов минимизированного автомата на экран.

Контрольные вопросы

1. Что такое состояния, совместимые по выходам?
2. Что такое полностью совместимые состояния?
3. Что такое таблица частичной совместимости?
4. Что такое таблица полной совместимости?
5. Что такое класс максимальной совместимости?
6. Расскажите все этапы алгоритма Пола-Ангера?
7. Что является предпосылками для минимизации конечного автомата?
8. Какие ещё алгоритмы минимизации автоматов вы знаете?
9. Является ли минимизированный автомат, полученный в результате работы алгоритма Пола-Ангера, всегда минимальным?
10. Зачем нужно минимизировать конечный автомат?
11. Другие вопросы по теме.